챕터 2. 비밀번호를 맡기지 않는 로그인. OAuth 2.0 & OIDC

이번 챕터가 끝나면

- 남의 서비스 비밀번호를 직접 받으면 왜 위험한지 이해합니다 (자격 위임 문제)
- 카카오 로그인이 일어나는 순서를 직접 따라가 봅니다 (Authorization Code 흐름)
- Access Token과 ID Token이 어떻게 다른지 구분해서 씁니다 (OAuth vs OIDC)

이번 챕터는 카카오 로그인을 붙이는 이야기입니다

챕터 1에서 `docker compose up` 한 줄로 띄운 Spring Boot 서비스 위에, 이제 로그인을 얹습니다. 자체 회원가입 폼을 직접 만들지 않고, 카카오에 신분 확인을 맡기는 쪽으로 갑니다. 그 과정에서 **"비밀번호를 내가 받아도 되는가"** 라는 질문이 먼저 등장하고, 그 답으로 OAuth 2.0의 인가 코드 방식이 나옵니다. 실습은 Spring Security를 직접 쓰지 않고 RestTemplate으로 카카오 서버와 대화해서, 한 요청이 어디로 튀고 무엇과 바꾸어지는지를 눈으로 따라갑니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 2의 실습 레포는 두 개입니다. 시작용과 완성용을 같이 열어 두고, 막히면 완성용의 해당 커밋을 펼쳐 보는 방식으로 진행합니다.

레포	용도	주소
kakao-oauth-code-ssr-start	카카오 로그인 기본 환경 (빈 자리 + Mustache 뷰)	`github.com/metacoding-11-spring-reference/kakao-oauth-code-ssr-start`

kakao-oauth-code-ssr-end	단계별 커밋이 담긴 완성 코드	`github.com/metacoding-11-spring-reference/kakao-oauth-code-ssr-end`

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/kakao-oauth-code-ssr-start.git
cd kakao-oauth-code-ssr-start
```

시작 코드는 이미 로그인 페이지(Mustache)와 컨트롤러·서비스 뼈대만 있는 상태입니다. 이번 챕터를 따라가며 비어 있는 메서드를 채워 나갑니다. 각 단계마다 `kakao-oauth-code-ssr-end`의 대응 커밋(`1.env`, `2.Authorization-code`, `3.Access-Token`, `4.user-session`)을 열어서 diff를 확인하면 됩니다.

2. 카카오 개발자 앱 만들기

카카오 로그인을 붙이려면, 내 서비스를 카카오 쪽에 먼저 등록해 두어야 합니다. 카카오 입장에서는 "이 주소로 오는 로그인 요청이 누구 것인지"를 알아야 하기 때문입니다.

[CAPTURE NEEDED: 카카오 디벨로퍼 사이트(https://developers.kakao.com/) 메인 화면. 상단 "내 애플리케이션" 진입 경로가 보이게]

1. `https://developers.kakao.com/` 로 이동해 카카오 계정으로 로그인합니다
2. 상단 **내 애플리케이션 → 애플리케이션 추가하기**로 새 앱을 만듭니다
3. 생성된 앱을 클릭해 **제품 설정 → 카카오 로그인 → 일반**으로 들어갑니다
4. **사용 설정**을 ON, **OpenID Connect**도 ON으로 바꿉니다
5. 같은 화면 아래에서 **Redirect URI**에 `http://localhost:8080/oauth/callback`을 등록합니다

[CAPTURE NEEDED: 카카오 로그인 일반 설정 화면. "사용 설정 ON"과 "OpenID Connect ON" 토글이 둘 다 켜진 상태]

[CAPTURE NEEDED: Redirect URI 입력란에 http://localhost:8080/oauth/callback 이 등록된 화면]

그다음 **제품 설정 → 카카오 로그인 → 동의항목**으로 이동해 **닉네임** 하나만 필수 동의로 바꿉니다. 닉네임이 있어야 유저 테이블에 이름을 넣을 수 있습니다.

[CAPTURE NEEDED: 동의항목 화면. 닉네임이 "필수 동의"로 체크된 상태]

마지막으로 앱 → 일반 → 플랫폼 키에서 REST API 키와, 같은 화면의 Client Secret 두 값을 메모장에 복사해 둡니다. 다음 절에서 `.env`에 넣을 값입니다.

3. 사용할 구성 요소

챕터 2에 새로 얹히는 재료는 세 가지뿐입니다. 하나는 카카오의 엔드포인트, 두 개는 Spring 쪽 클래스입니다.

재료	역할
카카오 인가 엔드포인트 (`kauth.kakao.com/oauth/authorize`)	사용자를 카카오 로그인 화면으로 보낸다
카카오 토큰 엔드포인트 (`kauth.kakao.com/oauth/token`)	인가 코드를 Access Token / ID Token으로 바꾼다
`RestTemplate` + `JWKSet` (nimbus-jose-jwt)	토큰 교환·ID Token 서명 검증

Spring Security를 쓰지 않는 이유

실무에서는 `spring-boot-starter-oauth2-client`를 쓰는 편이 빠르지만, 이번 챕터에서는 일부러 직접 HTTP로 카카오와 대화합니다. 자동화된 라이브러리가 뒤에서 대신 쳐 주던 단계(인가 요청 URL 조립 → code 수신 → token 교환 → JWKS 조회 → 서명 검증)를 한 번씩 눈으로 보고 나면, 나중에 Security로 옮겨 갈 때 어떤 콜백이 무엇을 하는지 감이 잡힙니다.

4. 실습 순서

이번 챕터의 절은 이 순서로 진행합니다.

1. `2.1`. 비밀번호를 직접 받으면 무엇이 문제인지 체감
2. `2.2`. OAuth 2.0의 네 주인공과 Authorization Code 방식 이해
3. `2.3`. 카카오 로그인 URL 만들고 인가 코드 받기
4. `2.4`. 인가 코드로 Access Token 교환, 사용자 정보 조회
5. `2.5`. OIDC로 업그레이드. ID Token 한 장으로 사용자 정보 끝내기
6. `2.6`. REST 방식·Credential 방식과 비교 정리

1에서 문제를 만난 뒤 2에서 비유로 길을 잡고, 3~5에서 한 단계씩 토큰을 손에 쥐어 봅니다. 6은 시야를 한 번 넓히는 정리 절입니다.

2.1 비밀번호를 내가 왜 받고 있지

챕터 1을 끝낸 다음 날 아침.

전날 `docker compose up` 한 줄로 서버를 띄워 둔 터라, 오픈이의 마음은 가벼웠습니다. 모니터 옆에 다 마신 커피잔이 두 개 올라와 있었습니다. 팀장은 오전 미팅 사이에 잠깐 들러 메신저에 한 줄을 남기고 돌아갔습니다.

팀장 : "이 서비스에 로그인 붙일 건데요. 카카오로 하죠."

카카오로.

오픈이는 일단 아무 생각 없이 설계를 그려 보기 시작했습니다. 로그인이면, 회원가입 폼 하나에 아이디와 비밀번호 칸이 필요할 테고, 뒤에는 User 테이블이 있을 테고, 로그인 시에는 입력한 비밀번호와 DB에 저장된 값을 비교하면 되는 거였습니다. BCrypt로 해시해서 넣고, 소금값도 붙이고, 세션에 사용자 id를 꽂아 두면 끝. 이건 학원에서 몇 번 해 본 흐름이었습니다.

그러다 팀장이 지나가는 말에 "카카오로"라고 했던 게 떠올랐습니다.

아, 카카오 아이디와 비번을 받아 오면 되는 건가.

손끝이 잠깐 멈췄습니다. 뭐가 이상했습니다. 옆자리 동료가 의자를 반쯤 돌려 화면을 봤습니다.

동료 : "지금 뭐 짜고 있어요? 설마 카카오 비번 받는 폼 그리는 건 아니죠?"

오픈이 : "아니에요. 아니... 잠깐만요."

동료 : "카카오 비번을 우리 서버가 받으면, 저희가 그걸 DB에 저장해요, 아님 그때그때 카카오한테 물어봐요?"

그러게.

질문을 듣고 나서야 문제가 보였습니다. 만약 카카오 비밀번호를 오픈이의 서비스 폼에 치게 한다면, 그 비밀번호는 이 서버를 한 번은 지나갑니다. 어딘가에 잠깐이라도 머뭅니다. 로그에 찍힐 수도 있고, 실수로 DB에 저장될 수도 있고, HTTP 요청이 중간에 가로채일 수도 있었습니다. 무엇보다 **카카오는 오픈이의 서비스를 신뢰하지 않습니다.** 카카오 입장에서는, 어디의 누가 만든 건지도 모르는 이 서비스에 자기 회원의 비밀번호가 그냥 흘러가는 걸 반길 리 없었습니다.

[GEMINI PROMPT: 한 사람(고객)이 왼쪽에 서 있고, 오른쪽에는 큰 건물(카카오)이 있다. 중앙에 작은 가게(우리 서비스)가 있다. 고객이 가게 직원에게 "카카오 비밀번호"라고 적힌 종이를 내밀고 있고, 가게 직원은 당혹스러운 표정. 배경 멀리 있는 카카오 건물 창문에서 "그 비번은 나한테만 주세요" 라는 말풍선. 깔끔한 일러스트풍, 흰 배경.]

아, 그래서 구글 로그인이나 카카오 로그인 버튼을 누르면, 그 비밀번호 입력창이 우리 사이트가 아니라 카카오 쪽으로 넘어갔었구나.

그 장면이 떠올랐습니다. 어디서든 소셜 로그인을 누르면 잠깐 주소창이 바뀌었습니다. `kauth.kakao.com`이나 `accounts.google.com` 같은 곳으로 갔다가, 로그인을 마치면 원래 사이트로 돌아왔습니다. 비밀번호는 카카오 안에서만 입력됐습니다. 오픈이의 서비스는 단 한 번도 그 값을 보지 않았습니다.

오픈이 : "비번을 받으면 안 되겠네요. 그럼 제가 뭘 받는 거죠. 카카오한테서."

동료 : "글쎄요. 뭔가 증표 같은 거 받지 않을까요."

두 사람이 말하는 동안 팀장이 다시 지나가다가 한마디 얹었습니다.

팀장 : "한번 생각해봐요. 호텔에 청소업체를 부른다고 칩시다. 청소업체가 내 방에 들어와야 하는데, 내가 방 비번을 직접 주고 싶진 않죠. 그럼 프런트에서 어떻게 할까요."

프런트에서.

프런트가 청소업체한테 **카드키**를 내줄 거 같은데.

카드키가 머릿속에 그려졌습니다. 호텔 프런트 직원이 청소 담당 직원에게 카드키 한 장을 건넵니다. 그 카드키는 특정 방 문만 열립니다. 유효 시간이 지나면 자동으로 먹통이 됩니다. 나중에 이상한 일이 생기면, 프런트는 그 카드만 지워 버리면 됩니다. 진짜 방 비밀번호는 호텔 시스템 안에만 있고, 청소업체는 그걸 모릅니다.

오픈이는 수첩을 폈습니다.

— 생각 정리 —

1. 문제의 정체
   - 우리 서비스가 카카오 비밀번호를 직접 받으면 위험
   - 받아서 저장해도 문제, 저장 안 해도 경로상 노출 문제
   - 카카오 입장에서도 내 회원 비번을 남이 들고 있는 걸 허용 못함
2. 해결 방향
   - 비번 대신 임시 통행증 같은 걸 받는다
   - 통행증은 카카오가 직접 만들어 준다
   - 통행증에는 "이 사람이 누구이고 무엇을 허락했는지"만 담겨 있다
3. 이 구조에 이름이 있을 것
   - 아마 OAuth 같은 것

호텔 카드키.

팀장이 말한 방향이 보였습니다. 청소업체는 방 비밀번호를 몰라도 됐습니다. **프런트가 대신 확인해 주고, 제한된 권한만 담긴 카드를 내주면** 됐습니다. 오픈이의 서비스도 같은 자리에 서면 됐습니다. 카카오 비밀번호는 카카오 안에서 처리되고, 오픈이의 서비스는 카카오가 발급해 준 카드 한 장만 들고 있으면 됩니다.

이 구조에 이름이 있는 모양이었습니다.

2.2 OAuth 2.0. 네 주인공과 교환권

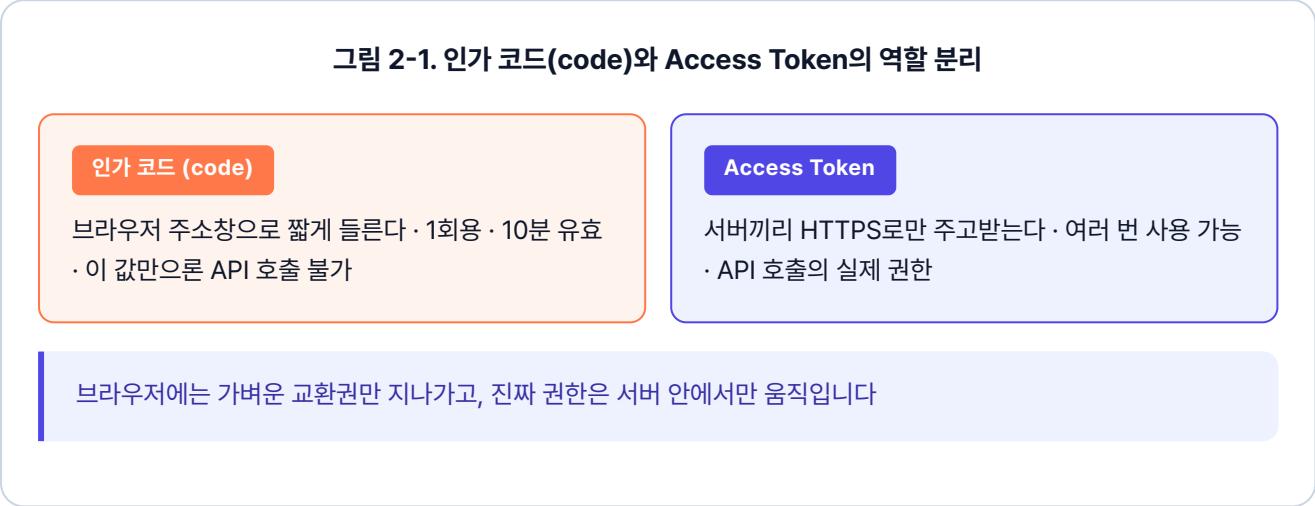
오픈이가 찾아본 이름은 **OAuth 2.0**이었습니다. 호텔 카드키 비유를 그대로 옮기면, OAuth는 다음 네 역할로 짜여 있었습니다.

[GEMINI PROMPT: 네 캐릭터가 각자의 자리에 서 있는 다이어그램. 왼쪽에 "사용자"(Resource Owner, 방의 주인). 가운데에 "우리 서비스"(Client, 청소업체). 오른쪽 상단에 "카카오 인증 서버"(Authorization Server, 호텔 프런트). 오른쪽 하단에 "카카오 API 서버"(Resource Server, 고객 방). 각 캐릭터 아래에 작은 한글 라벨. 깔끔한 개념도, 흰 배경, 인디고·오렌지 악센트.]

간단히 말해, **사용자**(카카오 계정의 주인)는 자기 정보의 접근을 **우리 서버**에 위임하고 싶어합니다. 위임을 확인해 주고 통행증을 발급하는 쪽이 **카카오 인증 서버**(Authorization Server)이고, 그 통행증으로 실제 데이터(프로필·닉네임)를 꺼낼 수 있는 곳이 **카카오 API 서버**(Resource Server)입니다. 오픈이의 Spring

서버는 한가운데에서 사용자를 대신해 뛰어다니는 **클라이언트**(Client)입니다.

이 네 명이 주고받는 **핵심 규칙**이 하나 있었습니다. 사용자가 카카오에서 로그인에 성공해도, 오픈이의 서비스가 곧바로 통행증(Access Token)을 받지는 않습니다. 먼저 **일회용 교환권**(Authorization Code)을 받고, 그 교환권을 다시 카카오 인증 서버에 가지고 가서 통행증으로 바꿉니다. 이렇게 **두 번 나누는 이유**가 있었습니다.



교환권은 브라우저 주소창(URL)을 지나갑니다. 주소창은 조심해도 새어나갈 여지가 있는 자리입니다. 기록에 남고, 옆 사람이 볼 수도 있고, 히스토리에 남을 수도 있습니다. 그래서 거기에 실제 권한을 실어 보내지 않고 **교환권만** 흘립니다. 그 교환권을 받은 오픈이의 서버가, 사용자가 보지 못하는 뒤편에서 카카오 서버에 **HTTPS POST 요청**을 한 번 더 날려 Access Token을 받아 옵니다. 이게 Authorization Code 방식의 핵심이었습니다.

브라우저엔 교환권. 서버끼리만 통행증.

2.3 카카오 로그인 URL 만들기. 인가 코드 받기

이제 코드를 열 차례였습니다. 먼저 받아야 할 게 교환권(code)이니, 오픈이의 서버가 할 일은 딱 하나였습니다. **사용자를 카카오 로그인 페이지로 데려다 놓는 것.**

2.3.1 환경변수 등록 (.env, application.properties)

`kakao-oauth-code-ssr-end`의 `1.env` 커밋을 참고해, 2.0절에서 복사해 둔 값 두 개를 프로젝트 루트의 `.env`에 넣습니다.

`.env`를 열고 아래 형식으로 작성합니다.

실습 1 .env. REST API 키 / Client Secret 저장

```
KAKAO_CLIENT_ID="여기에 REST API 키 붙여넣기"
KAKAO_CLIENT_SECRET="여기에 Client Secret 붙여넣기"
```

`.env`는 **깃에 올리지 않습니다.** 레포의 `.gitignore`에 이미 `.env`가 들어 있어 자동으로 제외됩니다. 같이 보는 `application.properties`는 카카오 엔드포인트 주소만 담아 둡니다.

`src/main/resources/application.properties`를 열고, 카카오 관련 줄을 확인합니다.

실습 2 application.properties. 카카오 엔드포인트 등록

```
# 카카오 OAuth 2.0 인가 요청 주소
# 사용자가 "카카오 로그인" 버튼을 누르면 이 주소로 브라우저가 이동
kakao.authorize-uri=https://kauth.kakao.com/oauth/authorize

# 카카오 로그인 완료 후 인가 코드(code)를 받을 우리 서버 주소
# 카카오 디벨로퍼의 Redirect URI와 동일해야 함
kakao.redirect-uri=http://localhost:8080/oauth/callback

# 인가 코드를 Access Token으로 바꿀 때 호출하는 주소
kakao.token-uri=https://kauth.kakao.com/oauth/token

# 사용자 정보 조회 API
kakao.user-info-uri=https://kapi.kakao.com/v2/user/me

# 환경변수 주입(.env 참조)
kakao.client-id=${KAKAO_CLIENT_ID}
kakao.client-secret=${KAKAO_CLIENT_SECRET}
```

`kakao.client-id`와 `kakao.client-secret`이 `.env`에서 주입되는 자리입니다. `${KAKAO_CLIENT_ID}` 문법을 쓰면 Spring Boot가 시작할 때 `.env`를 읽어 값을 대입합니다. 이렇게 분리해 두면, 키는 커밋 이력에 남지 않고 엔드포인트만 추적할 수 있습니다.

속성	값	출처
`kakao.authorize-uri`	`https://kauth.kakao.com/oauth/authorize`	카카오 공식 문서
`kakao.redirect-uri`	`http://localhost:8080/oauth/callback`	카카오 디벨로퍼에 등록한 값과 동일해야 함

`kakao.token-uri`	`https://kauth.kakao.com/oauth/token`	카카오 공식 문서
`kakao.client-id`	`${KAKAO_CLIENT_ID}`	`.env`에서 주입
`kakao.client-secret`	`${KAKAO_CLIENT_SECRET}`	`.env`에서 주입

2.3.2 로그인 페이지와 리다이렉트

사용자가 `/login` 페이지에서 "카카오로 로그인하기"를 누르면 어떤 일이 일어나는지 그림으로 먼저 봅니다.

그림 2-2. 인가 코드가 오픈이 서버까지 도착하는 여섯 걸음

그림의 여섯 걸음 중에서, 오픈이가 코드로 짜야 할 부분은 두 지점뿐입니다. **(3) 인가 URL을 만들어서 리다이렉트하는 컨트롤러**와 **(7) 돌아온 code를 받는 콜백 컨트롤러**. 나머지는 브라우저와 카카오가 알아서 처리합니다.

`src/main/resources/templates/login.mustache`는 이미 준비되어 있습니다. 카카오 로그인 링크 한 줄만 확인합니다.

참고 src/main/resources/templates/login.mustache

```
<section>
    <h2>카카오 로그인</h2>
    <a href="/login/kakao">카카오로 로그인하기</a>
</section>
```

`a href="/login/kakao"`를 누르면 오픈이의 서버에 GET 요청이 날아갑니다. 그 요청을 받는 쪽은 `UserController`입니다.

`src/main/java/com/metacoding/spring_oauth/user/UserController.java`를 열고 `redirectToKakao()` 메서드의 `// TODO:` 자리를 아래 코드로 채웁니다.

실습 3 UserController.java. 카카오 인가 URL로 리다이렉트

```
@GetMapping("/login/kakao")
public String redirectToKakao() {
```

```
    // 1. 서비스에서 인가 요청 URL을 만든다
    // 2. "redirect:" 접두어로 브라우저를 그 URL로 보낸다
    return "redirect:" + userService.카카오로그인주소();
}
```

컨트롤러는 메서드 하나에 세 줄입니다. `카카오로그인주소()`가 돌려준 전체 URL 앞에 `redirect:`를 붙여서 반환하면 Spring MVC가 **302 Found** 응답을 보내 브라우저를 해당 주소로 이동시킵니다. 이 한 번의 리다이렉트로 사용자의 브라우저는 오픈이의 서버를 떠나 카카오로 넘어갑니다.

그다음 실제 URL을 조립하는 서비스 메서드입니다. `UserService.java`를 열고 주입 필드 세 개가 있는지 확인합니다.

참고 UserService.java. 환경변수 주입 필드

```
@Value("${kakao.authorize-uri}")
private String kakaoAuthorizeUri;

@Value("${kakao.client-id}")
private String kakaoClientId;

@Value("${kakao.redirect-uri}")
private String kakaoRedirectUri;
```

`@Value`는 `application.properties`에서 값을 받아 필드에 꽂아 주는 Spring 애너테이션입니다. 앞 절에서 등록한 세 값이 여기에 들어옵니다.

같은 파일의 `카카오로그인주소()` 메서드 TODO 자리를 채웁니다.

실습 4 UserService.java. 인가 코드 요청 URL 조립

```
public String 카카오로그인주소() {
    // 1. redirect_uri는 URL 안에 중첩으로 들어가니 인코딩
    String encodedRedirect = URLEncoder.encode(
            kakaoRedirectUri, StandardCharsets.UTF_8);

    // 2. 쿼리 파라미터 네 개를 이어 붙여 최종 URL 완성
    return kakaoAuthorizeUri
            + "?response_type=code"
            + "&client_id=" + kakaoClientId
            + "&redirect_uri=" + encodedRedirect
            + "&scope=profile_nickname";
}
```

`response_type=code`는 "나는 인가 코드 방식을 쓰겠다"는 선언입니다. `client_id`는 카카오 앱 식별자, `redirect_uri`는 카카오가 `code`를 돌려 줄 우리 서버 주소, `scope=profile_nickname`은 "닉네임 동의만 받겠다"는 요청입니다.

`redirect_uri`는 URL 한가운데에 또 다른 URL이 들어가는 자리라, `http://`의 `/`와 `:`가 그대로 들어가면 쿼리스트링 파싱이 깨집니다. `URLEncoder.encode`는 이 특수문자를 `%3A`·`%2F` 같은 퍼센트 인코딩 형태로 바꿔서, 카카오 서버가 한 덩어리의 값으로 안전하게 해석하도록 합니다.

이 메서드가 돌려주는 URL은 아래와 같은 모양입니다.

완성된 인가 코드 요청 URL (예시)

```
https://kauth.kakao.com/oauth/authorize
 ?response_type=code
 &client_id=4c9035da98c350e01a23cfeeab4a0f4a
 &redirect_uri=http%3A%2F%2Flocalhost%3A8080%2Foauth%2Fcallback
 &scope=profile_nickname
```

서버를 실행해 `http://localhost:8080/login`에 접속한 뒤 "카카오로 로그인하기"를 눌러 보면, 브라우저 주소창이 `kauth.kakao.com`으로 바뀌면서 카카오 로그인 화면이 뜹니다.

터미널 실험 2-1 실행. Spring 서버 기동

```
./gradlew bootRun
```

[CAPTURE NEEDED: 브라우저에서 http://localhost:8080/login 화면 → "카카오로 로그인하기" 클릭 → kauth.kakao.com의 카카오 로그인 입력 화면으로 넘어간 스크린샷]

[CAPTURE NEEDED: 카카오 로그인을 마친 뒤 뜨는 동의 화면. "프로필 정보(닉네임)" 동의 항목이 체크되어 있음]

2.3.3 콜백에서 인가 코드 받기

사용자가 동의 버튼을 누르면, 카카오는 등록해 둔 `redirect_uri`(=`/oauth/callback`)로 브라우저를 **다시 보냅니다.** 그 주소 뒤에 `?code=...` 쿼리가 붙어 있습니다. 이 code를 받는 컨트롤러를 붙입니다.

실습 5 UserController.java. 콜백에서 code 수신

```
@GetMapping("/oauth/callback")
public String kakaoCallback(@RequestParam("code") String code) {
    // 1. 받은 인가 코드를 로그로 확인 (이 단계에서는 임시)
    System.out.println("code : " + code);

    // 2. 일단 홈 화면으로 리다이렉트
    return "redirect:/";
}
```

`@RequestParam("code")`는 쿼리스트링에서 `code=...` 값을 꺼내 파라미터에 꽂아 줍니다. 지금은 이 값을 그냥 표준 출력으로 찍어만 두고 다음 절에서 쓸 예정입니다.

서버를 다시 띄우고 카카오 로그인을 한 번 더 해 봅니다.

[CAPTURE NEEDED: IntelliJ 콘솔에 "code : abcdef123..." 형태의 인가 코드가 찍힌 스크린샷]

콘솔에 긴 문자열이 하나 찍혔다면 교환권을 손에 쥔 겁니다. 이 값 자체로는 카카오 API를 부를 수 없습니다. 다음 절에서 이걸 **Access Token**으로 바꿉니다.

인가 코드 실험에서 생각해 볼 것들

- **code는 일회용입니다**: 한 번 토큰과 교환하면 끝. 같은 code로 두 번 요청하면 카카오가 `KOE320` 같은 에러를 내려 줍니다. 같은 브라우저로 로그인 흐름을 반복해 보면 code는 매번 달라집니다
- **code는 10분 안에 써야 합니다**: 교환권은 짧은 시간만 유효합니다. 카카오 공식 문서 기준 10분. 디버깅 중 잠깐 멈췄다가 다시 요청하면 만료된 경우가 많습니다
- **Redirect URI는 문자열 단위로 동일해야 합니다**: 카카오 디벨로퍼에 `http://localhost:8080/oauth/callback`을 등록해 놓고 코드에선 슬래시 하나를 더 붙이면 `KOE004` 불일치 에러가 납니다
- **인가 요청은 GET이고, 토큰 교환은 POST입니다**: 권한을 받는 과정과 실제 권한을 획득하는 과정의 HTTP 메서드가 다릅니다. GET은 사용자 브라우저가, POST는 우리 서버가 수행합니다

2.4 토큰으로 바꾸고 사용자 정보 받기

교환권을 손에 쥐었으니, 이제 통행증과 바꿀 차례입니다. `kakao-oauth-code-ssr-end`의 `3.Access-Token` · `4.user-session` 커밋을 따라갑니다.

2.4.1 토큰 응답 DTO 만들기

카카오가 `POST /oauth/token` 응답으로 내려주는 JSON을 받을 record를 만듭니다.

`src/main/java/com/metacoding/spring_oauth/user/KakaoResponse.java`를 열고 `TokenDTO`를 확인합니다.

참고 KakaoResponse.java. 토큰 응답 DTO

```
public class KakaoResponse {
    public static record TokenDTO(
            @JsonProperty("token_type") String tokenType,
            @JsonProperty("access_token") String accessToken,
            @JsonProperty("expires_in") Long expiresIn,
            @JsonProperty("refresh_token") String refreshToken,
            @JsonProperty("refresh_token_expires_in") Long refreshTokenExpiresIn,
            String scope) {
    }
}
```

`@JsonProperty`는 카카오가 내려주는 snake_case 필드명(`access_token`)을 자바의 camelCase 필드(`accessToken`)로 연결해 주는 애너테이션입니다. Jackson이 역직렬화할 때 이 대응을 읽습니다. 자바 record는 불변 DTO를 만드는 가장 짧은 방법입니다.

2.4.2 토큰 교환 클라이언트 (RestTemplate)

실제 HTTP를 쏘는 부품은 `KakaoApiClient`입니다.
`src/main/java/com/metacoding/spring_oauth/_core/utils/KakaoApiClient.java`를 열고 TODO 자리에 아래 코드를 채웁니다.

실습 6 KakaoApiClient.java. 인가 코드 → Access Token 교환

```java
public KakaoResponse.TokenDTO getKakaoToken(String code) {
    // 1. 토큰 엔드포인트에 보낼 POST 요청(헤더+바디)을 구성
    HttpEntity<MultiValueMap<String, String>> request = createTokenRequest(code);

    // 2. RestTemplate으로 동기 POST 호출
    ResponseEntity<KakaoResponse.TokenDTO> response = restTemplate.exchange(
            kakaoTokenUri,
            HttpMethod.POST,
            request,
            KakaoResponse.TokenDTO.class);

    // 3. 응답 바디(TokenDTO)만 반환
    return response.getBody();
}

private HttpEntity<MultiValueMap<String, String>> createTokenRequest(String code) {
    HttpHeaders headers = new HttpHeaders();
    headers.add("Content-Type", "application/x-www-form-urlencoded;charset=utf-8");

    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("grant_type", "authorization_code");
    body.add("client_id", kakaoClientId);
    body.add("redirect_uri", kakaoRedirectUri);
    body.add("code", code);
    if (StringUtils.hasText(kakaoClientSecret)) {
        body.add("client_secret", kakaoClientSecret);
    }

    return new HttpEntity<>(body, headers);
}
```

`MultiValueMap`으로 바디를 조립하는 이유는 카카오가 `Content-Type: application/x-www-form-urlencoded`를 요구하기 때문입니다. JSON이 아닙니다. 네 가지 필드(`grant_type` · `client_id` · `redirect_uri` · `code`)가 필수이고, 앱 설정에서 Client Secret이 켜져 있으면 `client_secret`도 함께 실어 줍니다.

바디 필드	값	역할
`grant_type`	`authorization_code`	"나는 인가 코드를 교환하러 왔다"는 고정값
`client_id`	환경변수	어떤 앱의 요청인지 식별

`redirect_uri`	환경변수	처음 인가 요청 때와 동일해야 함
`code`	콜백에서 받은 값	방금 쥔 교환권
`client_secret`	환경변수 (선택)	앱 설정에서 "사용함"이면 필수

서비스에서 이 메서드를 불러 주기만 하면 토큰이 손에 들어옵니다. `UserService.java`의 `카카오로그인()` 메서드를 먼저 1차로 채웁니다.

실습 7 UserService.java. 토큰 교환만 확인하는 1차 버전

```
@Transactional
public UserResponse.DTO 카카오로그인(String code) {
    // 1. 인가 코드를 카카오 토큰 엔드포인트에 가져가 Access Token과 교환
    KakaoResponse.TokenDTO tokenDTO = kakaoApiClient.getKakaoToken(code);

    // 2. 확인용 출력 (다음 단계에서 사용자 조회로 이어짐)
    System.out.println("tokenDTO = " + tokenDTO);
    return null;
}
```

콜백 컨트롤러도 이 메서드를 부르도록 한 줄 고칩니다.

실습 8 UserController.java. 콜백에서 서비스 호출

```
@GetMapping("/oauth/callback")
public String kakaoCallback(@RequestParam("code") String code) {
    userService.카카오로그인(code);
    return "redirect:/post/list";
}
```

서버를 다시 띄우고 로그인해 봅니다.

[CAPTURE NEEDED: 카카오 로그인 후 IntelliJ 콘솔에 "tokenDTO = TokenDTO[tokenType=bearer, accessToken=..., expiresIn=21599, refreshToken=..., scope=profile_nickname]" 찍힌 로그]

`tokenType=bearer` · `accessToken=...` · `expiresIn=21599`(약 6시간)· `refreshToken=...` 네 값이 찍혔다면, 방 문이 열리는 카드키가 실제로 발급된 겁니다. 브라우저에서는 `/post/list` 페이지로 넘어가지만, 아직 사용자 조회를 붙이지 않았기에 "사용자 조회가 되지 않았습니다" 같은 빈 화면이 보입니다.

2.4.3 사용자 정보 조회와 세션 저장

Access Token을 **Authorization 헤더**에 실어서 `kapi.kakao.com/v2/user/me`를 호출하면, 카카오가 사용자 정보를 내려줍니다. `KakaoApiClient`에 메서드 하나를 더 추가합니다.

실습 9 KakaoApiClient.java. Access Token → 사용자 정보

```
public KakaoResponse.KakaoUserDTO getKakaoUser(String accessToken) {
    // 1. Bearer 토큰을 헤더에 실어 GET 요청 구성
    HttpEntity<MultiValueMap<String, String>> request = createUserRequest(accessToke
n);

    // 2. /v2/user/me로 동기 GET
    ResponseEntity<KakaoResponse.KakaoUserDTO> response = restTemplate.exchange(
            kakaoUserInfoUri,
            HttpMethod.GET,
            request,
            KakaoResponse.KakaoUserDTO.class);

    return response.getBody();
}

private HttpEntity<MultiValueMap<String, String>> createUserRequest(String accessToke
n) {
    HttpHeaders headers = new HttpHeaders();
    headers.add("Content-Type", "application/x-www-form-urlencoded;charset=utf-8");
    headers.add("Authorization", "Bearer " + accessToken);
    return new HttpEntity<>(headers);
}
```

포인트는 `Authorization: Bearer {accessToken}` 한 줄입니다. "이 카드키로 문 열겠습니다"라는 요청입니다. 응답 DTO도 `KakaoResponse.java`에 미리 만들어 둡니다.

참고 KakaoResponse.java. 사용자 정보 DTO 추가

```
public static record KakaoUserDTO(
        Long id,
        @JsonProperty("connected_at") Timestamp connectedAt,
        Properties properties) {
}

public static record Properties(String nickname) {
}
```

이제 `카카오로그인()` 메서드를 DB 저장까지 이어지는 최종 형태로 고칩니다.

실습 10 UserService.java. 사용자 조회 + DB 저장 (SSR 최종)

```
@Transactional
public UserResponse.DTO 카카오로그인(String code) {
    // 1. 인가 코드로 토큰 요청
    KakaoResponse.TokenDTO token = kakaoApiClient.getKakaoToken(code);

    // 2. Access Token으로 카카오 유저 조회
    KakaoResponse.KakaoUserDTO kakaoUser =
            kakaoApiClient.getKakaoUser(token.accessToken());

    // 3. 닉네임을 username으로 삼아 DB에 upsert
    String username = kakaoUser.properties().nickname();
    User user = userRepository.findByUsername(username)
            .orElseGet(() -> userRepository.save(
                    User.builder()
                            .username(username)
                            .password(UUID.randomUUID().toString())
                            .email("kakao" + kakaoUser.id() + "@kakao.com")
                            .provider("kakao")
                            .build()));

    return new UserResponse.DTO(user);
}
```

`password`에 `UUID.randomUUID()`를 넣는 건, 이 계정은 카카오가 인증하므로 **우리 서버에서는 절대로 로그인에 사용되지 않는 비밀번호**라는 뜻입니다. 로컬 로그인 경로 자체를 만들지 않을 것이므로 이 값은 평생 쓰이지 않습니다. `provider="kakao"`는 "이 사용자는 카카오로 들어왔다"는 출처 표시입니다.

마지막으로 컨트롤러에서 세션에 사용자를 꽂습니다.

실습 11 UserController.java. 세션 저장 (SSR 최종)

```
@GetMapping("/oauth/callback")
public String kakaoCallback(@RequestParam("code") String code) {
    UserResponse.DTO sessionUser = userService.카카오로그인(code);
    session.setAttribute("sessionUser", sessionUser);
    return "redirect:/post/list";
}
```

서버를 띄우고 다시 로그인하면, 이번에는 `/post/list`에서 닉네임이 찍힌 화면이 나옵니다.

[CAPTURE NEEDED: /post/list 페이지 상단에 "환영합니다, {닉네임}님" 같은 사용자 정보가 표시된 스크린샷]

호텔 카드키가 실제로 문을 연 순간이었습니다. **비밀번호는 단 한 번도 오픈이의 서버를 지나가지 않았습니다.** 카카오 로그인 화면에서 사용자가 입력한 비번은 카카오 안에만 머물렀고, 오픈이의 서버가 받은 건 code 한 줄, Access Token 한 장, 사용자 정보 JSON 하나가 전부였습니다.

2.5 한 걸음 더. OIDC로 ID Token 한 장에 끝내기

로그인이 붙었지만, 팀장이 퇴근 전에 한마디를 더 던지고 갔습니다.

팀장 : "REST API로도 만들어 놓고 싶은데, 그때는 사용자 조회 한 번 더 가는 게 낭비예요. 토큰 안에 정보가 들어 있으면 안 부르면 되거든요. OIDC 한번 찾아봐요."

OIDC.

OAuth 2.0은 "권한 위임"을 표준화한 규칙이었습니다. 누가 누구의 데이터를 얼마나 만질 수 있는지를 정하는 쪽입니다. 로그인 자체는 엄밀히 말해 그 위에 얹은 관행이었습니다. Access Token을 받아서 `/v2/user/me`를 부르는 건 **OAuth의 본 의도와는 조금 결이 다른** 활용법입니다.

그래서 그 위에 **OIDC**(OpenID Connect)가 얹혔습니다. 정의는 다음과 같이 바꿔 볼 수 있습니다.

- **OAuth 2.0**: "이 사용자가 자기 데이터 중 X를 우리 서비스에 허용했다"는 **권한 증표**(Access Token) 발급 프로토콜
- **OIDC**: OAuth 위에 "이 사용자가 누구인지"를 서명된 JWT로 같이 실어 주는 **신원 증명** 프로토콜

카카오 입장에서 Access Token은 "방 청소해도 됩니다"라고 적힌 카드키였고, OIDC의 ID Token은 "이 사람은 3456789012345678번 회원이고 닉네임은 xxx이고 10분 안엔 확실히 유효합니다"라고 **카카오가 직접 도장 찍어서 주는 신분증**이었습니다. 도장은 **RSA 서명**으로 찍혀 있어서, 카카오의 공개키로만 검증할 수 있습니다.

2.5.1 ID Token의 구조

카카오가 `id_token` 필드에 담아서 내려주는 JWT는 세 조각입니다.

id_token 포맷 예시

```
Header
{
  "alg": "RS256",
  "kid": "16b-key-id-2025-01",
  "typ": "JWT"
}

Payload (클레임)
{
  "iss": "https://kauth.kakao.com",
  "aud": "101234567890",
  "sub": "345678901234567",
  "exp": 1734988000,
  "iat": 1734984400,
  "nickname": "오픈이"
}

Signature
RS256 서명 바이트열
```

- `iss` (Issuer): 누가 발급했는가(카카오)
- `aud` (Audience): 누구에게 발급했는가(우리 앱의 client_id)
- `sub` (Subject): 이 토큰의 주인(카카오 회원의 고유 id)
- `exp` · `iat` : 만료 시각·발급 시각
- `nickname` : 동의 항목에 포함시키면 함께 실림

서명은 카카오의 **비밀키**로 찍혀 있고, 오픈이 서버는 카카오의 **공개키**로 검증합니다. 공개키 묶음은 `https://kauth.kakao.com/.well-known/jwks.json` 에서 조회할 수 있습니다. 이 묶음이 **JWKS**(JSON Web Key Set)입니다.

2.5.2 OIDC 활성화와 scope 변경

OIDC 사용 전에 2.0절에서 켜 둔 **OpenID Connect ON**이 되어 있는지 다시 확인합니다. 그다음 `UserService` 의 `카카오로그인주소()` 에서 scope에 `openid` 를 추가합니다.

실습 12 UserService.java. scope에 openid 추가 (OIDC REST 버전)

```
public String 카카오로그인주소() {
    String encodedRedirect = URLEncoder.encode(
            kakaoRedirectUri, StandardCharsets.UTF_8);

    // scope에 "openid"가 들어가야 카카오가 id_token을 함께 발급
    String scope = URLEncoder.encode("openid profile_nickname",
            StandardCharsets.UTF_8);

    return kakaoAuthorizeUri
            + "?response_type=code"
            + "&client_id=" + kakaoClientId
            + "&redirect_uri=" + encodedRedirect
            + "&scope=" + scope;
}
```

`scope`에 `openid`가 빠져 있으면 카카오는 Access Token만 주고, `id_token`은 빈값으로 내려옵니다. `profile_nickname`은 그대로 유지해서 클레임에 닉네임을 받습니다.

그리고 `KakaoResponse.TokenDTO`에 `id_token` 필드를 추가합니다.

실습 13 KakaoResponse.java. TokenDTO에 idToken 필드 추가

```
public record TokenDTO(
        @JsonProperty("token_type") String tokenType,
        @JsonProperty("access_token") String accessToken,
        @JsonProperty("expires_in") Long expiresIn,
        @JsonProperty("refresh_token") String refreshToken,
        @JsonProperty("refresh_token_expires_in") Long refreshTokenExpiresIn,
        @JsonProperty("id_token") String idToken,
        String scope) {
}
```

2.5.3 JWKS로 서명 검증하기

핵심은 `KakaoOidcUtil`입니다. 이 클래스가 하는 일은 세 가지입니다. JWT 파싱, JWKS에서 kid에 맞는 공개키 조회, 서명 검증 + 클레임 추출.

`src/main/java/com/metacoding/spring_oauth_oidc/_core/utils/KakaoOidcUtil.java`의 TODO 자리에 아래 코드를 채웁니다.

실습 14 KakaoOidcUtil.java. ID Token 서명 검증과 클레임 추출

```java
public KakaoOidcResponse verify(String idToken) {
    try {
        // 1. 문자열 JWT를 SignedJWT 객체로 파싱
        SignedJWT signedJWT = SignedJWT.parse(idToken);

        // 2. 헤더의 kid로 카카오 JWKS에서 맞는 공개키 조회
        RSAKey rsaKey = getKeyFromJwks(signedJWT.getHeader().getKeyID());

        // 3. RSA 공개키로 서명 검증
        if (!signedJWT.verify(new RSASSAVerifier(rsaKey))) {
            throw new RuntimeException("카카오 id_token 서명 검증 실패");
        }

        // 4. 검증 통과 후 클레임 꺼내기
        JWTClaimsSet claims = signedJWT.getJWTClaimsSet();
        return new KakaoOidcResponse(
                claims.getSubject(),
                claims.getStringClaim("nickname"),
                claims.getExpirationTime().toInstant());

    } catch (ParseException | JOSEException e) {
        throw new RuntimeException("카카오 id_token 검증 중 오류 발생", e);
    }
}

private RSAKey getKeyFromJwks(String keyId) {
    try {
        // 1. 카카오 JWKS JSON 전체를 내려받아 파싱
        JWKSet jwkSet = JWKSet.load(URI.create(kakaoOidcJwksUri).toURL());

        // 2. id_token 헤더의 kid와 일치하는 키를 고름
        JWK jwk = jwkSet.getKeyByKeyId(keyId);
        if (!(jwk instanceof RSAKey rsaKey)) {
            throw new RuntimeException("RSA 공개키가 아닙니다: " + keyId);
        }
        return rsaKey;
    } catch (Exception e) {
        throw new RuntimeException("JWKS 조회 실패", e);
    }
}
```

핵심은 **kid 기반 조회**입니다. 카카오는 여러 키 쌍을 돌려 가며 씁니다(키 로테이션). ID Token 헤더의 `kid`(key id)에 어느 키로 서명했는지가 박혀 있으므로, JWKS에서 그 kid를 가진 키만 뽑아 검증에 씁니다. 서명이 맞으면 `claims.getSubject()`(=카카오 회원 id)와

`claims.getStringClaim("nickname")`만 꺼내면 끝. `/v2/user/me`**를 부르지 않습니다.**

nimbus-jose-jwt 구성요소	역할
`SignedJWT.parse`	문자열 JWT를 파싱해 헤더·페이로드·서명 3분할
`JWKSet.load(URL)`	카카오 JWKS JSON을 다운로드·파싱
`RSASSAVerifier`	RS256 서명 검증기 (공개키로 서명 확인)
`JWTClaimsSet`	검증 후 클레임(`sub`·`nickname`·`exp`)을 꺼내는 접근자

그리고 REST 버전의 `UserService.카카오로그인()`은 이렇게 짧아집니다.

실습 15 UserService.java. OIDC REST 버전 최종

```
@Transactional
public UserResponse.DTO 카카오로그인(String code) {
    // 1. 인가 코드 → Access Token + ID Token 동시 수신
    KakaoResponse.TokenDTO tokenDTO = kakaoApiClient.getKakaoToken(code);

    // 2. ID Token 서명 검증하며 사용자 클레임 추출 (/v2/user/me 호출 없음)
    KakaoOidcResponse resDTO = kakaoOidcUtil.verify(tokenDTO.idToken());

    // 3. DB 저장 + 우리 서비스 JWT 발급
    User user = 카카오유저생성및갱신(resDTO.subject(), resDTO.nickname());
    String jwt = JwtUtil.create(user);

    return new UserResponse.DTO(user, jwt);
}
```

SSR 버전과 비교하면 **사용자 조회 API 호출 한 번이 사라졌습니다.** 토큰 안에 이미 사용자 식별자(`sub`)와 닉네임이 들어 있으니 카카오에 한 번 더 물어볼 이유가 없습니다.

OIDC 실험에서 생각해 볼 것들

- **scope에 openid가 빠지면 id_token은 빈값입니다**: 카카오는 `scope=openid`가 있어야만 OIDC 응답을 내려줍니다. 같은 앱을 OIDC로도·아닌 채로도 쓸 수 있게 설계되어 있습니다

- **kid가 바뀌면 JWKS를 다시 받아와야 합니다**: 카카오가 키를 로테이션하면 기존 kid는 JWKS에서 사라집니다. 실무에서는 JWKS를 캐시하되 만료 시간을 짧게 잡고, miss가 나면 다시 받아오는 정책이 필요합니다
- **서명 검증만으로 충분하지 않을 때도 있습니다**: `iss`(발급자)가 `https://kauth.kakao.com`이 맞는지, `aud`(수신자)가 내 client_id가 맞는지, `exp`가 지났는지를 함께 검사하면 더 안전합니다. 현재 코드는 서명까지만 수행합니다
- **우리 서비스 JWT는 카카오 ID Token과 따로 둡니다**: 카카오 ID Token은 "카카오가 확인해 준 신원"이고, `JwtUtil.create`로 발급하는 우리 서비스 JWT는 "우리 서버가 이 사용자를 로그인 상태로 인정한다"는 분리된 토큰입니다. 다음 챕터에서 이 우리 서비스 토큰을 Redis에 공유할 세션과 함께 다룹니다

2.6 REST, OIDC, Credential. 한눈에 정리

오픈이는 하루 만에 세 가지 이름을 구분할 수 있게 됐습니다. 세 가지가 다 OAuth 2.0 가족이지만, 쓰이는 환경과 토큰 흐름이 다릅니다.

그림 2-3. 세 가지 방식 비교

Code (SSR)	브라우저로 code 수신 → 서버가 Access Token 교환 → `/v2/user/me`로 사용자 조회. 2.3~2.4에서 다룬 흐름
Code + OIDC (REST)	scope에 openid 추가 → id_token까지 함께 수신 → JWKS로 서명 검증하여 사용자 정보 추출. 2.5에서 다룬 흐름
Credential (SDK)	Flutter·iOS·Android 앱에서 카카오 SDK로 바로 로그인 → access_token / id_token을 앱이 받아 서버로 전달 → 서버는 유효성만 검증

> 웹은 주로 Code/OIDC, 앱은 주로 Credential. 서버가 검증하는 대상만 달라집니다

Credential 방식은 이 책의 실습 범위(Spring 서버)는 아니지만, 앱 개발자와 같이 일할 때 자주 등장하므로 한 단락만 더 붙여 둡니다. 앱이 카카오 SDK로 로그인하면, SDK가 네이티브로 로그인 화면을 띄우고 토큰을 받아 옵니다. 웹에서 브라우저가 왔다 갔다 하던 리다이렉트가 앱 안에서 보이지 않게 처리됩니다. 서버 쪽의 할 일은 하나입니다. 앱이 보낸 `access_token`이나 `id_token`이 **카카오가**

진짜 발급한 것이 맞는지 검증하는 것. Access Token이면 `GET kapi.kakao.com/v1/user/access_token_info`로 카카오에 물어보고, ID Token이면 **이번 챕터 2.5에서 만든 JWKS 검증**을 그대로 재사용하면 됩니다.

로그인 흐름 하나에 이름이 세 개나 붙는 이유가 있었구나.

오픈이는 화면을 한 번 껐다가 켰습니다. 닉네임이 떠 있는 `/post/list` 페이지가 보였습니다. 어제 `docker compose up`으로 띄운 빈 `hello world!`에서, 이제는 카카오로 로그인한 사용자가 세션에 들어가 있는 서비스로 한 걸음 올라간 셈입니다.

오후에 동료가 다시 의자를 돌려 화면을 봤습니다.

동료 : "로그인 붙었네요. 근데 저도 제 노트북에서 똑같이 로그인해 봤는데요, 제 세션이 자꾸 끊어져요. 서버를 두 번 껐다 켰더니요."

오픈이 : "세션이 서버 메모리에 있으니까요. 껐다 켜면 날아가요."

동료 : "그럼 서버 두 대 띄우면 어떻게 돼요? 한 대에서 로그인한 사람이 다른 대로 넘어가면 또 풀리는 거예요?"

아.

오픈이는 잠깐 멈췄습니다. 지금까지는 Docker 컨테이너 하나 위에서만 움직였습니다. 만약 스케일 아웃이 필요해서 컨테이너를 두 개 띄우면, 왼쪽 컨테이너에서 찍은 `session.setAttribute("sessionUser", ...)` 값이 오른쪽 컨테이너에는 없을 터였습니다. 로그인한 사용자가 요청마다 다른 컨테이너로 날아갈 수도 있을 텐데, 그때마다 새로 로그인하라고 할 수는 없었습니다.

오픈이는 수첩 한쪽에 다시 물음표를 하나 그렸습니다.

세션을 컨테이너 바깥에 둘 수 있는 곳.

다음 챕터의 문이 열리고 있었습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
호텔 프런트	Authorization Server (인증 서버)	사용자 로그인과 동의를 처리하고 토큰을 발급하는 서버. 예: `kauth.kakao.com`
고객 방	Resource Server (자원 서버)	사용자 데이터가 저장된 서버. Access Token을 실은 요청만 처리. 예: `kapi.kakao.com`
청소업체	Client (클라이언트)	사용자를 대신해 자원 서버에 접근하는 애플리케이션. 여기서는 Spring 서버
방의 주인	Resource Owner (자원 소유자)	데이터의 실제 소유자. 사용자 본인
일회용 교환권	Authorization Code (인가 코드)	사용자 동의를 증명하는 1회용·단기 유효 값. 브라우저 URL로 전달
호텔 카드키	Access Token	자원 서버 API를 호출할 수 있는 권한 증표. Authorization 헤더에 `Bearer` 접두어로 실어 사용
카카오가 발급한 신분증	ID Token	OIDC에서 발급되는 서명된 JWT. 사용자 신원 정보(`sub`·`nickname` 등)와 서명 포함
공개키 묶음	JWKS (JSON Web Key Set)	ID Token 서명을 검증할 때 쓰는 공개키들의 집합. 카카오는 `https://kauth.kakao.com/.well-known/jwks.json`로 제공
서명 식별자	kid (Key ID)	ID Token 헤더에 박히는 서명 키 식별자. JWKS에서 같은 kid를 가진 공개키로 검증

권한 위임 프로토콜	OAuth 2.0	사용자가 제3자 서비스에 자기 자원 접근 권한을 안전하게 위임하는 표준 프로토콜
신원 확인 프로토콜	OIDC (OpenID Connect)	OAuth 2.0 위에 얹은 표준. 권한 위임이 아니라 "이 사용자가 누구인가"를 ID Token으로 증명
SDK 로그인	Credential 방식	모바일 앱 SDK가 로그인과 토큰 수신을 내부에서 처리하고, 서버는 토큰 유효성만 검증

이것만은 기억하자

- **우리 서버는 남의 비밀번호를 받지 않는다.** 카카오 비번은 카카오 화면에서만 입력되고, 우리 서버가 받는 건 일회용 교환권(code)이다
- **교환권과 통행증은 흐르는 길이 다르다.** code는 브라우저 URL로, Access Token은 서버끼리 POST로만 주고받는다
- **OAuth는 권한, OIDC는 신원.** Access Token이면 `/v2/user/me`를 한 번 더 호출하고, ID Token이면 JWKS로 서명만 검증하면 사용자 정보까지 끝난다
- **다음 챕터에서는** 세션을 서버 메모리 대신 **Redis**에 올려서, 컨테이너가 죽었다 살아나도·여러 대로 늘어나도 로그인 상태가 끊기지 않도록 만들어 봅니다 (Redis 세션 공유)